



RAFFLES INSTITUTION

Quick Reference Guide

1. Python

1. Identifiers

When naming variables, functions and modules, the following rules must be observed:

- Names should begin with character 'a' - 'z' or 'A' - 'Z' or '_' and followed by alphanumeric characters or '_'.
- Reserved words should not be used.
- User-defined identifiers are case sensitive.

2. Comments and Documentation Strings

```
# This is a comment
```

```
"""
    This is a documentation string
    over multiple lines
"""
```

3. Input/Output

```
print ("This is a string")
```

```
s = input ("Instructions to prompt for data
entry.")
```

4. Import

```
import <module>
```

```
from <module> import <name>
```

5. Data Type

Data Type	Notes
int	integer
float	real number
bool	boolean
str	string (immutable)
list	series of values
dict	key-value pairs
tuple	series of values (immutable)

6. Assignment

Assignment Statement	Notes
a = 1	integer
b = c	variable
d = "This is a string"	string
mylist = [1, 2, 3, 4, 5]	list
mydict = {'key': 'value'}	Dict

7. Arithmetic Operators

Operator	Notes
+ -	plus, subtract
* /	multiply, divide
%	remainder or modulus
**	exponential or power
//	quotient of the floor division

8. Relational Operators

Operator	Notes
==	equality
!=	not equal to
> >=	greater than, greater than or equal to
< <=	less than, less than or equal to

9. Boolean Expression

Boolean Expression	Notes
a and b	logical and
a or b	logical or
not a	logical not

10. Iteration

while loop	for loop
while condition(s): <statement(s)>	for i in range(n): <statement(s)>
	for record in records: <statement(s)>

11. Selection

Type 1	Type 2	Type 3
<pre>if condition(s): <statement(s)></pre>	<pre>if condition(s): <statement(s)> else: <statement(s)></pre>	<pre>if condition(s): <statement(s)> elif condition(s): <statement(s)> else: <statement(s)></pre>

12. Functions

Function definitions

@<optional decorator(s)>

```
def <function name> (<parameters>):
    <function body>
```

Function calls

<function name>(<value>, <name>=<value>)

13. Object-Oriented Programming**class** <class name> (<optional parent class>):

```
def __init__(self, <parameters>):
    <constructor body>
```

```
def <method name> (self, <parameters>):
    <method body>
```

14. Built-in Functions and Attributes

<code>__file__</code>	<code><file>.readlines()</code>	<code><list>.copy()</code>	<code>print()</code>	<code><str>.isdigit()</code>
<code>__name__</code>	<code><file>.write()</code>	<code><list>.index()</code>	<code>range()</code>	<code><str>.islower()</code>
<code>abs()</code>	<code>float()</code>	<code><list>.insert()</code>	<code>round()</code>	<code><str>.isspace()</code>
<code>bin()</code>	<code>hex()</code>	<code><list>.pop()</code>	<code>staticmethod()</code>	<code><str>.isupper()</code>
<code><bytes>.decode()</code>	<code>input()</code>	<code><list>.remove()</code>	<code>str()</code>	<code><str>.lower()</code>
<code>chr()</code>	<code>int()</code>	<code><list>.reverse()</code>	<code><str>.encode()</code>	<code><str>.startswith()</code>
<code><dict>.clear()</code>	<code>len()</code>	<code><list>.sort()</code>	<code><str>.endswith()</code>	<code><str>.upper()</code>
<code><dict>.copy()</code>	<code>list()</code>	<code>max()</code>	<code><str>.format()</code>	
<code><file>.close()</code>	<code><list>.append()</code>	<code>min()</code>	<code><str>.index()</code>	
<code><file>.read()</code>	<code><list>.extend()</code>	<code>open()</code>	<code><str>.isalnum()</code>	
<code><file>.readline()</code>	<code><list>.clear()</code>	<code>ord()</code>	<code><str>.isalpha()</code>	

csv module	datetime module		math module
<code>reader()</code> <code>writer()</code> <code><writer>.writerow()</code>	<code>datetime()</code> <code>datetime.now()</code> <code>datetime.strptime()</code> <code><datetime>.isoformat()</code> <code><datetime>.strftime()</code> <code><datetime>.year</code> <code><datetime>.month</code>	<code><datetime>.day</code> <code><datetime>.hour</code> <code><datetime>.minute</code> <code><datetime>.second</code> <code><timedelta>.days</code> <code><timedelta>.seconds</code>	<code>ceil()</code> <code>exp()</code> <code>floor()</code> <code>log()</code> <code>pow()</code> <code>sqrt()</code> <code>trunc()</code>

os.path module	random module	sqlite3 module	socket module	sys module
<code>basename()</code> <code>dirname()</code> <code>isdir()</code> <code>isfile()</code> <code>join()</code>	<code>random()</code> <code>randint()</code> <code>randrange()</code> <code>shuffle()</code>	<code>connect()</code> <code><connection>.commit()</code> <code><connection>.close()</code> <code><connection>.execute()</code> <code><connection>.rollback()</code> <code><connection>.row_factory</code> <code><cursor>.fetchone()</code> <code><cursor>.fetchall()</code> <code>Row</code>	<code>socket()</code> <code>bind()</code> <code>listen()</code> <code>accept()</code> <code>connect()</code> <code>recv()</code> <code>sendall()</code>	<code>exit()</code>

15. Additional Functions and Attributes

pymongo module		flask module
MongoClient() <client>.database_names() <client>.get_database() <client>.drop_database() <client>.close() <database>.collection_names() <database>.get_collection() <database>.drop_collection() <collection>.insert_one() <collection>.insert_many() <collection>.find_one() <collection>.find()	<collection>.update_one() <collection>.update_many() <collection>.delete_one() <collection>.delete_many() <collection>.count() <cursor>.count()	Flask() <flask application>.route() <flask application>.run() render_template() request.files request.form request.method send_from_directory() redirect() url_for() secure_filename() <uploaded file>.save()

2. SQL Statements

```

CREATE TABLE table_name(
  column1_name COLUMN1_TYPE COLUMN1_CONSTRAINTS,
  column2_name COLUMN2_TYPE COLUMN2_CONSTRAINTS,
  ...
  PRIMARY KEY (column1_name, column2_name, ...),
  FOREIGN KEY (column_name) REFERENCES table_name(column_name)
);

```

```

SELECT column1_name, column2_name, ...
FROM table_name
WHERE where_expression
ORDER BY order_expression ASC;

```

```

SELECT column1_name, column2_name, ...
FROM table_name
WHERE where_expression
ORDER BY order_expression DESC;

```

```

SELECT table1_name.column1_name, table2_name.column2_name, ...
FROM table_name, table2_name
WHERE where_expression;

```

```

SELECT table1_name.column1_name, table2_name.column2_name, ...
FROM table1_name
INNER JOIN table2_name ON join_expression;

```

```

SELECT table1_name.column1_name, table2_name.column2_name, ...
FROM table1_name
LEFT OUTER JOIN table2_name ON join_expression;

```

```

SELECT
  COUNT(*),
  MAX(column1_name),
  MIN(column2_name),
  SUM(column3_name),
  ...
FROM table_name;

```

```

INSERT INTO table_name(column1_name, column2_name, ...)
VALUES(column1_value, column2_value, ...);

```

```

UPDATE table_name SET
  column1_name = column1_expression,
  column2_name = column2_expression,
  ...
WHERE where_expression;

```

```

DELETE FROM table_name
WHERE where_expression;

```

```

DROP TABLE table_name;

```

3. SQLite Types, Constraints, Functions and Operators

Types	Constraints	Functions	Operators			
NULL	NOT NULL	COUNT()		/	<	AND
REAL	PRIMARY KEY	MAX()	+	%	<=	OR
INTEGER	AUTOINCREMENT	MIN()	-	=	>	IS
TEXT	UNIQUE	SUM()	*	!=	>=	IS NOT

4. PyMongo Operators

Comparison

\$eq	\$gt	\$gte	\$lt	\$lte
\$ne	\$in	\$nin		

Logical

\$and	\$not	\$or
-------	-------	------

Element

\$exists

Update

\$set	\$unset
-------	---------

5. HTML Elements, Attributes and Character References

The first line of a HTML document must be: `<!doctype html>`

Type	Elements	Attributes
<i>Common</i>		id, class
<i>Required</i>	<code><html></code> , <code><head></code> , <code><title></code> , <code><body></code>	
<i>Metadata</i>	<code><link></code>	rel, href
<i>Structure</i>	<code><h1></code> , <code><h2></code> , <code><h3></code> , <code><p></code> , <code><div></code> , <code></code> , <code><hr></code>	
<i>Text and Media</i>	<code></code> , <code><i></code>	
	<code><a></code>	href
	<code></code>	src, alt
<i>Table</i>	<code><table></code> , <code><tr></code> , <code><th></code> , <code><td></code>	
<i>Form</i>	<code><form></code>	action, enctype, method
	<code><input></code>	name, type, value
	<code><textarea></code>	name

Character	&	<	>	"
Reference	<code>&amp;</code>	<code>&lt;</code>	<code>&gt;</code>	<code>&quot;</code>

6. Jinja2 Filters

length	safe
--------	------

7. CSS Properties

Common	Box Model		Typography
display background color	height width border border-bottom border-left border-right border-top margin margin-bottom	margin-left margin-right margin-top padding padding-bottom padding-left padding-right padding-top	font-family font-size font-style font-weight text-align text-decoration